PROVISIONAL PATENT APPLICATION

SYSTEM AND METHOD FOR PRIVACY-PRESERVING SEMANTIC DISCOVERY OF DISTRIBUTED DATA USING PSEUDONYMOUS VECTORIZED FRAGMENTS WITH TWO-STAGE ANONYMITY GATING

Authors: Tom Tang, Fei Jia
Organization: Edge Vector Foundation
Date: March 31, 2026

Applicant

Edge Vector Foundation
2328 Delmar Dr E, Seattle, WA 98102

Inventors

- Tom Tang, Seattle, WA
- Fei Jia, Kenmore, WA

FIELD OF THE INVENTION

This invention relates to privacy-preserving data discovery across distributed databases, and more particularly to a system and method for enabling users to discover that semantically similar data exists on other users' nodes without revealing the identity of the data owner, the structure of the source data, or the content of any individual record.

CROSS-REFERENCE TO RELATED APPLICATIONS

None (provisional filing).

BACKGROUND OF THE INVENTION

In distributed data systems where each user controls their own data on a local node, a fundamental challenge is enabling discovery — allowing users to find others who have similar or complementary data — without compromising the privacy of either party.

Existing approaches to data discovery fall into two categories:

1. **Centralized index systems** (e.g., search engines, data marketplaces) require users to upload metadata or data samples to a central server. This exposes data structure, field names, content, and identity to the index operator.
2. **Private set intersection (PSI) and related cryptographic protocols** enable two parties to discover overlapping records without revealing non-overlapping ones. However, PSI requires both parties to already know they want to query each other and scales poorly when searching across many nodes.

The concept of k-anonymity — ensuring that any individual record is indistinguishable from at least k-1 other records — was introduced for structured tabular data. However, k-anonymity has not been applied to vector embedding neighborhood density as a gating mechanism for publishing fragments to a discovery index. Existing k-anonymity systems operate on exact attribute values in relational tables, not on continuous-valued embedding vectors with cosine similarity thresholds.

Neither approach solves the fundamental discovery problem: how does a user discover that relevant data exists *somewhere* in the network, without knowing in advance where to look, and without exposing their own data or identity to the search infrastructure?

There is a need for a system that enables semantic discovery across a network of distributed, user-controlled databases while preserving the anonymity of data owners, preventing linkability of fragments from the same owner, and excluding personally identifiable information from the discovery index.

SUMMARY OF THE INVENTION

The present invention provides a system and method for privacy-preserving semantic discovery comprising:

1. **Per-field independent embedding** of user data, where each field value in a schema is embedded independently as a vector rather than concatenating all fields into a single embedding, enabling fine-grained control over which fields are published to the discovery index.

2. **A two-stage anonymity gate** that prevents identifying information from entering the discovery index:
   - **Stage 1 (Local, on-node):** A three-part filter comprising (a) field privacy classification that categorizes fields as NeverPublish, PublishIfAnonymous, or AlwaysPublish based on field name heuristics; (b) rule-based named entity recognition that detects PII patterns including email addresses, phone numbers, URLs, physical addresses, and identification numbers; and (c) token entropy analysis that rejects fragments with insufficient information entropy to ensure anonymity.
   - **Stage 2 (Network, on-server):** A k-anonymity check that measures the embedding neighborhood density of each incoming fragment using cosine similarity against existing fragments in the index, accepting only fragments that have at least k semantically similar neighbors (default $k = 50$, similarity threshold 0.85).
3. **Per-fragment unlinkable pseudonym derivation** using HMAC-based key derivation from the user's master signing key and the fragment's content hash, producing a unique Ed25519 keypair per fragment such that:
   - Two pseudonyms from the same user are computationally indistinguishable from random
   - The same user with the same content always derives the same pseudonym (enabling deduplication)
   - Different users with the same content derive different pseudonyms (preventing cross-user correlation)
   - The user can re-derive the signing key to prove ownership when responding to connection requests
4. **A quarantine and promotion system** where fragments that fail the network k-anonymity check are held in a staging table, invisible to search queries, and periodically re-evaluated as the network grows and embedding neighborhoods densify, with automatic promotion when the k-anonymity threshold is met and automatic expiration after a configurable period (default 90 days).
5. **An anonymous connection request protocol** where:
   - A searcher discovers semantically similar fragments identified only by pseudonym
   - The searcher submits a connection request addressed to a pseudonym with a one-time requester pseudonym and an optional message
   - The server looks up the target pseudonym's owner in a separate, access-controlled mapping table
   - The data owner receives the request with no information about the requester's identity
   - Acceptance reveals both parties' identities for data sharing negotiation; rejection reveals nothing

DETAILED DESCRIPTION OF THE INVENTION

System Architecture

The system comprises two principal components: (1) user-controlled database nodes that perform local anonymity checking and pseudonym derivation, and (2) a discovery service that maintains the searchable index and handles connection routing.

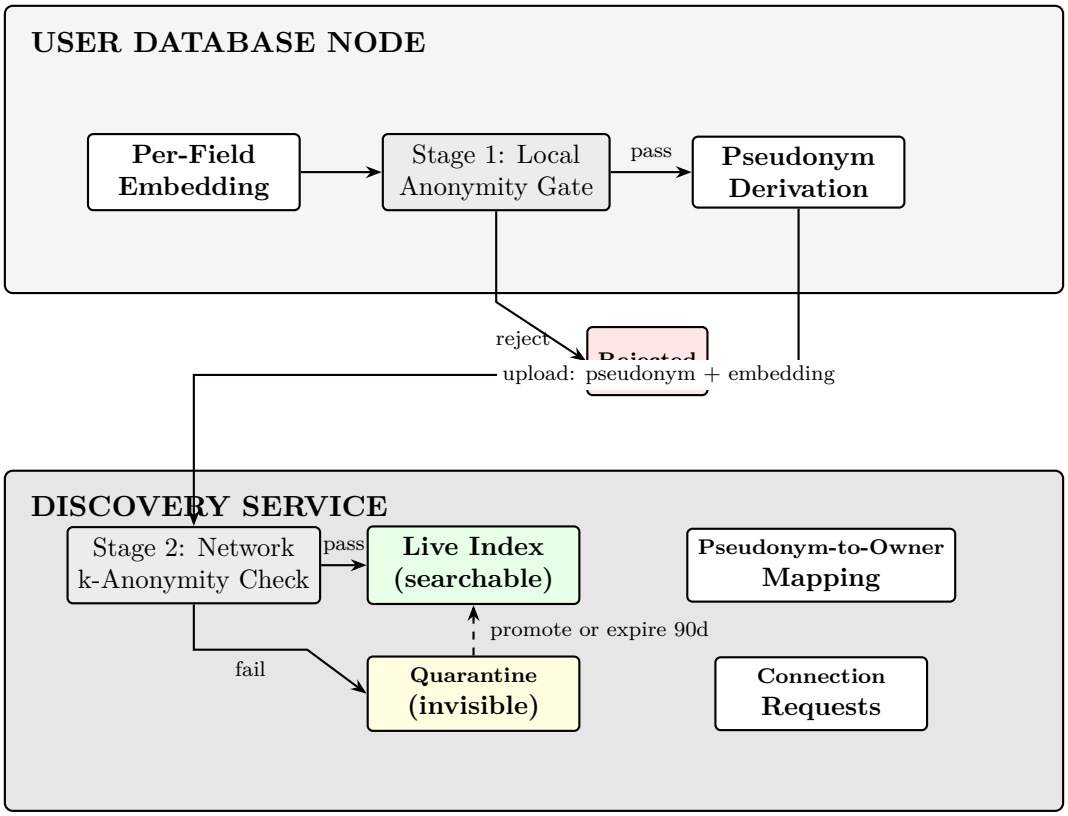

Figure 1: **System Architecture.** Two-component architecture: user database nodes (per-field embedding, local anonymity gate, pseudonym derivation) and discovery service (network k-anonymity check, live index, quarantine, connection routing).

Per-Field Independent Embedding

In contrast to prior systems that concatenate all field values into a single embedding per record, the present invention embeds each field value independently:

For each field in a schema record:

1. Extract the field value as text
2. If the text exceeds a length threshold (e.g., 100 characters), split into sentence-level fragments, each representing a coherent semantic unit
3. Generate an embedding vector (e.g., 384-dimensional) for each fragment independently

Storage keys include the schema name, record identifier, field name, and fragment index, enabling per-field granularity in the discovery index.

Content-type-specific splitting strategies include:

- Short text (under threshold): treated as a single fragment
- Long text (over threshold): split at sentence boundaries
- Structured/numeric values: treated as single fragments
- Arrays: each element treated as a separate fragment

This per-field approach enables fine-grained privacy control: users can publish embeddings for innocuous fields (e.g., recipe ingredients) while withholding sensitive fields (e.g., author name) from the same record.

Stage 1: Local Anonymity Gate

Every fragment must pass three local checks before submission to the network. These checks run entirely on the user's node with no network communication.

Check 1 — Field Privacy Classification: Each schema field is classified into one of three privacy classes:

- **NeverPublish:** Fields whose names match known PII patterns (name, email, phone, address, SSN, date of birth, IP address, username, URL, credit card, account number, passport, license). These fields are unconditionally excluded from the discovery index regardless of content.

- **AlwaysPublish:** Fields whose names match known low-cardinality, non-identifying categories (category, genre, type, cuisine, language, country, tags, format, status, priority, level). These fields are published without further checking.
- **PublishIfAnonymous:** All other fields. These proceed to checks 2 and 3.

Classification defaults are determined by pattern matching against the field name. Users may override the default classification for any field.

Check 2 — Named Entity Recognition: For PublishIfAnonymous fields, rule-based pattern matching detects PII that was not caught by field name classification:

- Email patterns (`word@domain.tld`)
- Phone patterns (sequences of 7+ digits with optional separators)
- URL patterns (`http(s)://` or `www.` prefixes)
- Physical address patterns (number + street name + street type suffix)
- Identification number patterns (SSN format, similar structured ID formats)

Rule-based detection is preferred over machine learning-based NER because it is deterministic, operates in microseconds without model loading, and errs conservatively (false positives are acceptable; false negatives — publishing PII — are not).

Fragments containing any detected entity are rejected.

Check 3 — Token Entropy Analysis: For fragments that pass NER, Shannon entropy over whitespace-delimited tokens is computed:

$$H = -\sum_i p_i \log_2(p_i)$$

where p_i is the frequency of token i divided by total token count.

Fragments with entropy below a minimum threshold (default 2.0 bits) are rejected as too specific or repetitive to be anonymous. Fragments shorter than a minimum word count (default 3 words) are also rejected.

Token entropy is a best-effort heuristic for anonymity, not a formal guarantee. A determined adversary could craft text with high entropy that is still identifying through other means. The entropy check is one layer in a defense-in-depth approach: field classification catches known PII fields, NER catches PII patterns in content, entropy catches low-diversity fragments, and the network k-anonymity check (Stage 2) provides the strongest guarantee by requiring semantic commonality across the user population.

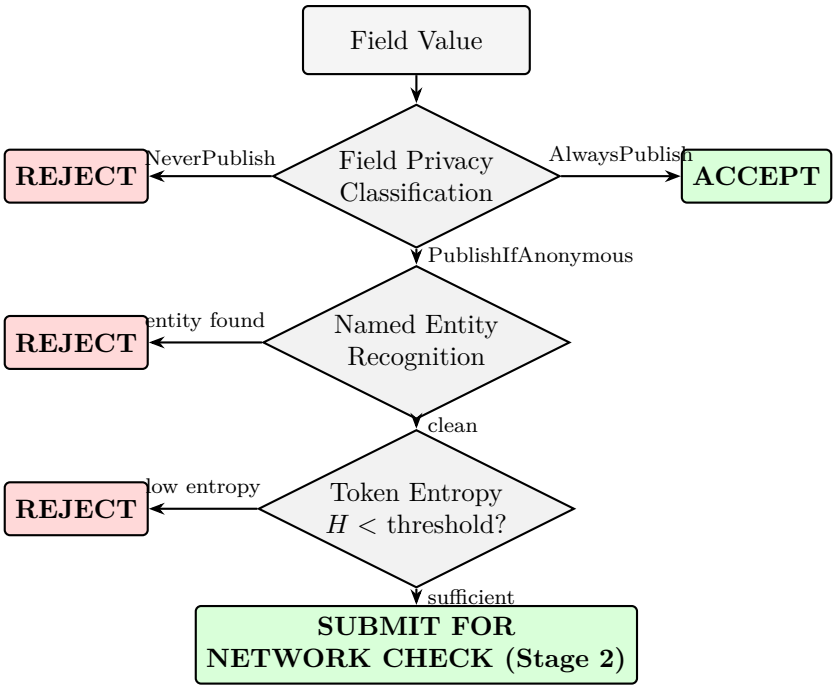

Figure 2: **Stage 1: Local Anonymity Gate.** Each field value passes through three sequential checks: field privacy classification, named entity recognition, and token entropy analysis. Only fragments that pass all three checks proceed to Stage 2 network checking.

Combined Decision: The three checks produce one of three decisions:

- **Accept** (AlwaysPublish fields)
- **Reject** with reason (NeverPublish, entity detected, insufficient entropy, too short)
- **SubmitForNetworkCheck** (passed all local checks, proceed to Stage 2)

Per-Fragment Pseudonym Derivation

Each fragment that passes Stage 1 receives a unique, unlinkable pseudonym before upload to the discovery index.

The derivation uses HMAC-based key derivation:

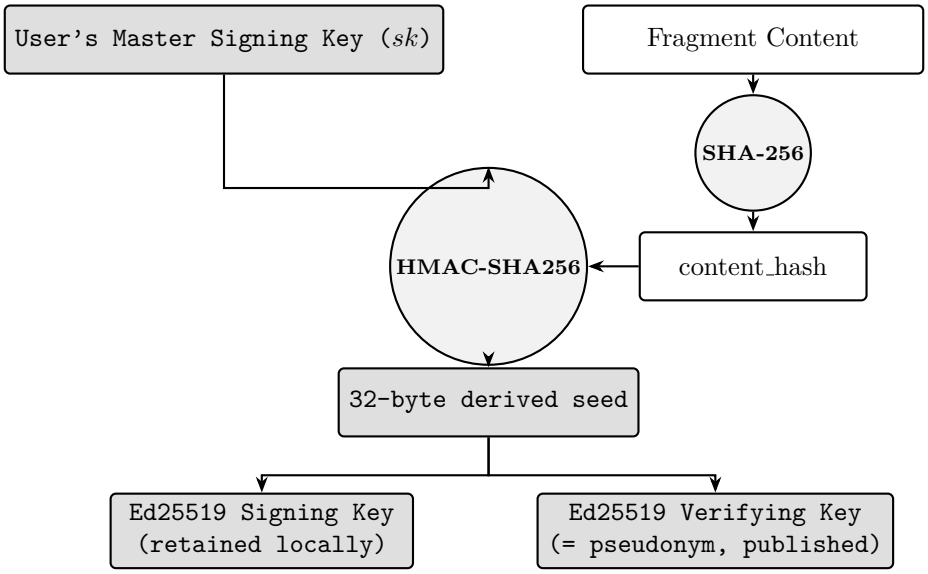

Figure 3: **Per-Fragment Pseudonym Derivation.** HMAC-SHA256 combines the user's master signing key with the SHA-256 hash of the fragment content to produce a 32-byte seed. This seed generates a unique Ed25519 keypair. The public key serves as the pseudonym; the signing key is retained locally for ownership proofs.

1. Compute SHA-256 hash of fragment content
2. Compute HMAC-SHA256 using the user's Ed25519 master signing key as the HMAC key and the content hash as the message
3. Use the resulting 32 bytes as an Ed25519 signing key seed
4. Derive the corresponding public key (verifying key)

The public key serves as the fragment's pseudonym identifier in the discovery index. The signing key is retained locally for proving ownership during connection requests.

Properties:

- **Deterministic:** Same master key + same content always produces the same pseudonym, enabling natural deduplication
- **Unlinkable:** Two pseudonyms from the same user are computationally indistinguishable from two pseudonyms from different users (HMAC output is pseudorandom)
- **Owner-verifiable:** The user can re-derive the signing key and produce a valid signature to prove ownership
- **Cross-user independent:** Different users with identical content produce different pseudonyms (different master keys)

Stage 2: Network k-Anonymity Check

When fragments arrive at the discovery service, the service measures embedding neighborhood density:

1. Compute cosine similarity between the incoming fragment's embedding and all existing fragments in the discovery index
2. Count the number of existing fragments with cosine similarity above a threshold (default 0.85)
3. If the count meets or exceeds the k-anonymity threshold (default $k = 50$), accept the fragment into the live discovery index
4. If the count is below threshold, quarantine the fragment in a staging table

The k-anonymity check ensures that published fragments are semantically common enough that any individual fragment cannot be attributed to a specific user by its content. A fragment about "chocolate cake recipes" is publishable because many users have similar data; a fragment about a unique proprietary process is quarantined because it would be uniquely identifying.

Cold-Start Behavior: At network launch, the discovery index is empty and no fragments meet the k-anonymity threshold. All initial fragments are quarantined. As users contribute fragments, embedding neighborhoods densify naturally. The quarantine promoter periodically re-evaluates staged fragments and promotes batches as neighborhoods reach the threshold. This means the index bootstraps itself organically — the first fragments to become searchable are the most common data categories (where many users contribute similar content), which are precisely the fragments safest to publish. The k-anonymity threshold may optionally be configured lower during an initial bootstrap period, with a monotonically increasing schedule toward the production threshold.

Scalability: At scale, exact cosine similarity computation against all existing fragments is replaced by approximate nearest-neighbor (ANN) search using hierarchical navigable small world (HNSW) graph indexing on the embedding vectors. HNSW provides sub-linear query time with tunable recall, enabling efficient neighborhood density estimation at millions or billions of fragments.

Quarantine and Promotion

Quarantined fragments are stored in a staging table that is invisible to search queries. A scheduled process (default: hourly) re-evaluates quarantined fragments:

1. For each quarantined fragment, recount the neighborhood size against the live discovery index
2. Fragments that now meet the k-anonymity threshold are promoted to the live index
3. Fragments quarantined beyond a maximum duration (default: 90 days) are expired and deleted

This mechanism ensures that the discovery index grows naturally: as more users contribute fragments, embedding neighborhoods densify, previously quarantined fragments meet the threshold, and the index becomes richer without compromising privacy.

Discovery Search

Search queries contain an embedding vector and optional filters (category, similarity threshold, result count). The discovery service performs nearest-neighbor search using HNSW (Hierarchical Navigable Small World) indexing on the embedding vectors.

Search results return only:

- Fragment pseudonym (UUID)
- Cosine similarity score
- Schema category (broad category like “recipes”, “health”, “finance”)
- Fragment type (field, sentence, element)
- Optional content preview (user-controlled length, may be empty)

Results never include user identity, schema name, field name, or any linkable identifier.

Anonymous Connection Request Protocol

The connection protocol is specifically designed to bridge the gap between anonymous discovery and identity-revealing data sharing. Unlike general-purpose anonymous messaging systems, it operates on the pseudonymous identifiers produced by the discovery index — the same pseudonyms that were derived through the anonymity gate and published to the vector index. This linkage between discovery and communication is the novel element: the searcher can contact an unknown data owner identified solely by the content of their published fragment, without either party revealing identity until mutual consent.

1. Searcher identifies a fragment of interest by its pseudonym

2. Searcher generates a one-time pseudonym for themselves
3. Searcher submits a connection request containing: target pseudonym, requester pseudonym, optional message
4. Discovery service looks up the target pseudonym in a separate access-controlled owner mapping table to determine the data owner's user hash
5. Connection request is stored with the owner's user hash as the routing key
6. Data owner polls for pending connection requests during regular sync
7. Owner sees the message and can Accept (revealing both identities for data sharing) or Reject (revealing nothing)

The owner mapping table (pseudonym to user hash) is access-controlled and never exposed to search queries. The connection request protocol reveals identities only upon mutual consent.

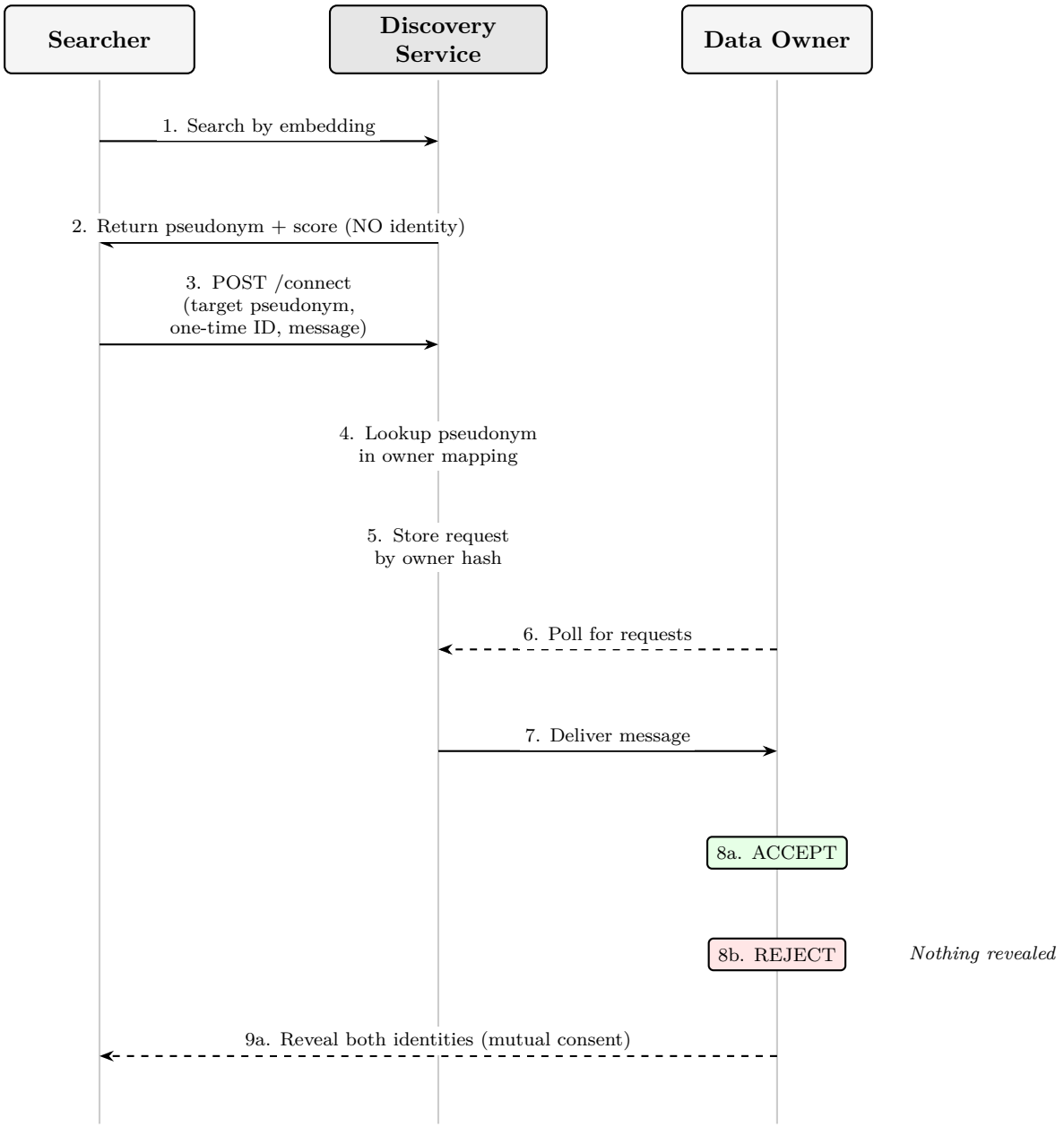

Figure 4: **Anonymous Connection Request Protocol.** A searcher discovers fragments by pseudonym, submits a connection request, and the discovery service routes it to the data owner. Identity disclosure occurs only upon mutual acceptance.

Trust Model for the Owner Mapping Table: The pseudonym-to-owner mapping table is the most sensitive component of the system, as its compromise would de-anonymize all published pseudonyms. The following mitigations apply:

1. **Access isolation:** The mapping table is stored in a separate database or partition from the searchable discovery index, with access restricted to the connection request handler. Search queries never join against or access the mapping table.
2. **Encryption at rest:** The mapping table is encrypted with a key managed by a hardware security module (HSM) or key management service, ensuring that database-level compromise does not expose the mappings.
3. **Minimal data retention:** The mapping table stores only the pseudonym UUID and the user's opaque hash identifier — no schema names, field names, or content. The user hash is itself a one-way derivative of the user's identity, adding an additional layer of indirection.
4. **Audit logging:** All accesses to the mapping table are logged with the requesting connection request ID, enabling detection of unauthorized access patterns.

Database Schema

The discovery service maintains four tables:

1. **discovery_fragments** (live index): pseudonym (UUID, primary key), embedding vector (384 dimensions), schema category, fragment type, content preview, opt-in timestamp. HNSW index on embedding vectors for efficient nearest-neighbor search.
2. **discovery_staging** (quarantine): same schema as `discovery_fragments` plus staging timestamp, last check timestamp, and current neighborhood size count.
3. **pseudonym_owners** (access-controlled mapping): pseudonym (UUID, primary key), user hash, schema name, field name, fragment index. Indexed by user hash for revocation queries.
4. **connection_requests** (routing): request ID, target user hash, target pseudonym, requester pseudonym, message, status (pending/accepted/rejected), timestamps.

Opt-Out and Revocation

Users can revoke discovery participation for any schema at any time:

1. User signs a revocation request with their master key

2. Discovery service verifies the signature
3. All fragments, staging entries, and owner mappings for that user and schema are deleted
4. Revocation is immediate and complete — no residual fragments remain in any table

CLAIMS

1. A computer-implemented method for privacy-preserving semantic discovery of data across distributed user-controlled databases, comprising:
    a. independently embedding each field value of a schema record as a separate vector, enabling per-field granularity in discovery index participation;
    b. applying a two-stage anonymity gate to each embedded fragment, wherein Stage 1 performs local on-node checks comprising field privacy classification, named entity recognition, and token entropy analysis, and Stage 2 performs a network k-anonymity check measuring embedding neighborhood density against existing fragments in the index;
    c. deriving a unique, unlinkable pseudonym for each fragment using HMAC-based key derivation from the user's master signing key and the fragment's content hash, such that pseudonyms from the same user are computationally indistinguishable from pseudonyms of different users;
    d. publishing accepted fragments to a searchable vector index identified only by pseudonym, with no user identity, schema name, or field name included in the searchable index;
    e. quarantining fragments that fail the k-anonymity check and periodically re-evaluating them for promotion as the network grows.
2. The method of claim 1, wherein the field privacy classification categorizes fields into three classes:
    a. NeverPublish fields identified by matching field names against known PII patterns, unconditionally excluded from the discovery index;
    b. AlwaysPublish fields identified by matching field names against known low-cardinality non-identifying categories, published without further checking;
    c. PublishIfAnonymous fields that proceed through named entity recognition and token entropy analysis before submission.
3. The method of claim 1, wherein the per-fragment pseudonym derivation:
    a. computes a content hash of the fragment text using SHA-256;
    b. applies HMAC-SHA256 using the user's Ed25519 master signing key as the HMAC key and the content hash as message;

c. uses the HMAC output as a seed for deriving an Ed25519 keypair;

d. uses the public key as the fragment's pseudonym identifier;

e. retains the signing key locally for proving ownership during connection requests;

such that the derivation is deterministic (enabling deduplication), unlinkable (two pseudonyms from the same user are indistinguishable from random), and cross-user independent (different users with identical content produce different pseudonyms).

4. The method of claim 1, wherein the network k-anonymity check:

   a. computes cosine similarity between the incoming fragment's embedding vector and all existing vectors in the discovery index;

   b. counts fragments exceeding a similarity threshold;

   c. accepts fragments with neighborhood count at or above a k-anonymity threshold into the live discovery index;

   d. quarantines fragments below the threshold into a staging table invisible to search queries;

   e. periodically re-evaluates quarantined fragments and promotes those that meet the threshold as the network densifies.

5. The method of claim 1, further comprising an anonymous connection request protocol wherein:

   a. a searcher identifies a fragment by its pseudonym from search results;

   b. the searcher submits a connection request containing a target pseudonym, a one-time requester pseudonym, and an optional message;

   c. the discovery service resolves the target pseudonym to the data owner's identity using an access-controlled mapping table not exposed to search queries;

   d. the data owner receives the connection request and may accept, revealing both parties' identities for data sharing negotiation, or reject, revealing nothing;

   such that identity disclosure is bilateral and consensual.

6. The method of claim 1, wherein the token entropy analysis computes Shannon entropy over whitespace-delimited tokens of the fragment text and rejects fragments with entropy below a configurable minimum threshold, ensuring that published fragments are semantically general rather than uniquely identifying.

7. The method of claim 1, wherein opt-out revocation for any schema is immediate and complete, comprising deletion of all fragments, staging entries, and pseudonym-to-owner mappings for the revoking user and specified schema, verified by a cryptographic signature from the user's master signing key.

8. The method of claim 1, wherein the pseudonym-to-owner mapping table is secured by:

a. storing the mapping table in a separate database or partition from the searchable discovery index, with access restricted to the connection request handler;

b. encrypting the mapping table at rest with a key managed by a hardware security module or key management service;

c. storing only the pseudonym UUID and the user's opaque hash identifier in the mapping table, with no schema names, field names, or content;

d. logging all accesses to the mapping table with the requesting connection request ID for audit trail purposes;

such that compromise of the searchable discovery index does not expose the pseudonym-to-owner mappings.

9. A system for privacy-preserving semantic discovery comprising:

a. a plurality of user-controlled database nodes, each configured to independently embed field values, apply local anonymity checks, derive per-fragment pseudonyms, and upload accepted fragments;

b. a discovery service comprising a vector database with HNSW indexing, a staging table for quarantined fragments, an access-controlled pseudonym-to-owner mapping table, and a connection request routing table;

c. a scheduled evaluation process that periodically re-checks quarantined fragments and promotes those meeting k-anonymity thresholds;

d. a search interface that returns fragments identified only by pseudonym, similarity score, broad category, and optional content preview, with no user identity or schema structure information.

ABSTRACT

A system and method for privacy-preserving semantic discovery across a network of distributed, user-controlled databases. Each user's database node independently embeds individual field values as vector fragments, applies a two-stage anonymity gate (local PII detection and entropy analysis followed by network k-anonymity checking), and derives a unique unlinkable pseudonym per fragment using HMAC-based key derivation. Accepted fragments are published to a centralized vector index identified only by pseudonym, with no user identity, schema name, or field information in the searchable index. Fragments failing k-anonymity are quarantined and periodically re-evaluated as the network grows. Users discover semantically similar data via vector similarity search and initiate anonymous connection requests that reveal identities only upon bilateral consent. The system enables discovery without surveillance: users can find that relevant data exists in the network without any party — including the discovery service operator — learning who owns what data.

Prepared for provisional patent filing by Edge Vector Foundation.
Filing date: [TO BE DETERMINED]
This document constitutes a provisional patent application under 35 U.S.C. 111(b).